TITLE

Assignment 3

TOPIC

Ultrasonic distance sensing: code, filters and map()

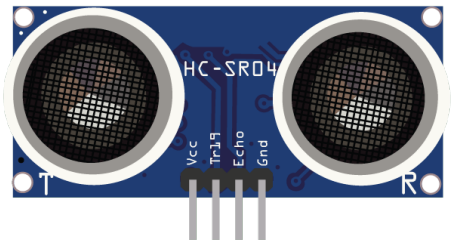
DEPARTMENT

IEEE SBUB · Supporting Document

FULL NAME

Youcef HARMOUNI

What this assignment is about



Turn an echo into a number.

1. Read the sensor with plain code, one line at a time
2. Learn the functions you need: `pulseIn`, `Serial`, `tone`, `noTone`, `map`
3. Clean noisy readings with 3 filtering methods
4. Scale 0 to 50 cm into 0 to 127

Your road map

1	2	3	4	5
Simple sensor	**Functions**	**Filters**	**map()**	**Assignments**
wire it, read it	what each one does	mean, median, smoothing	0..50 cm to 0..127	1 required, 1 extra

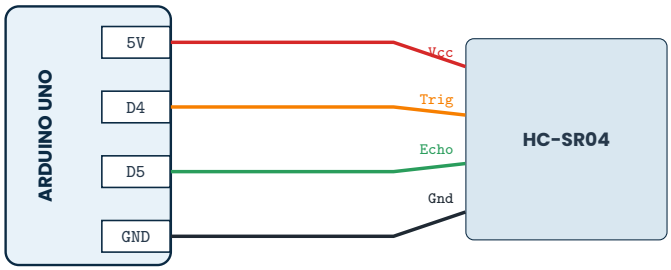
What you need

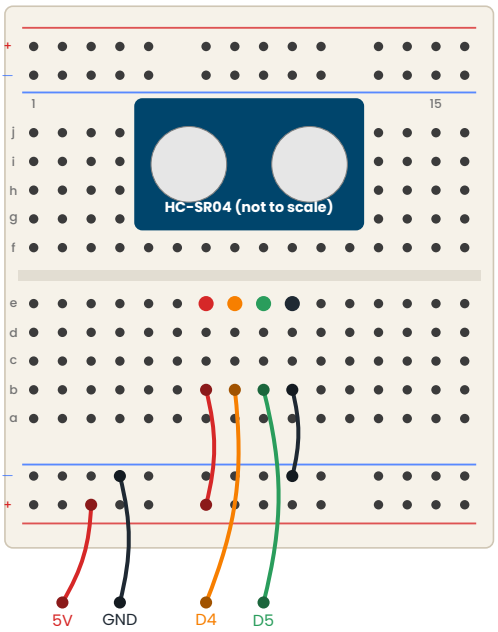
- 1 Arduino Uno + USB cable
- 1 HC-SR04 ultrasonic sensor
- 1 Optional: buzzer on pin 8 for the `tone()` demo
- 1 Breadboard
- 4 Jumper wires

1 Simple ultrasonic

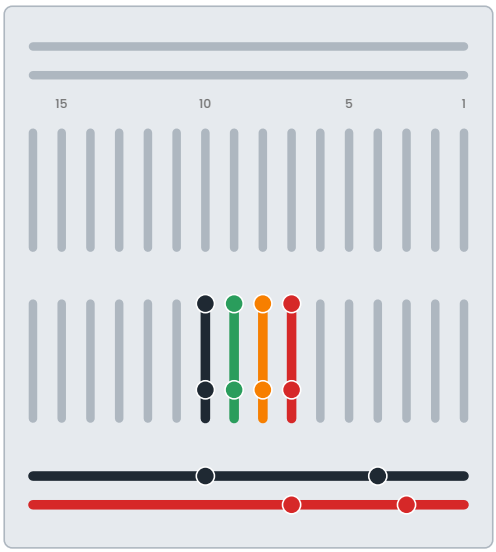
Wire it, then build the code one line at a time

1.1 Wiring





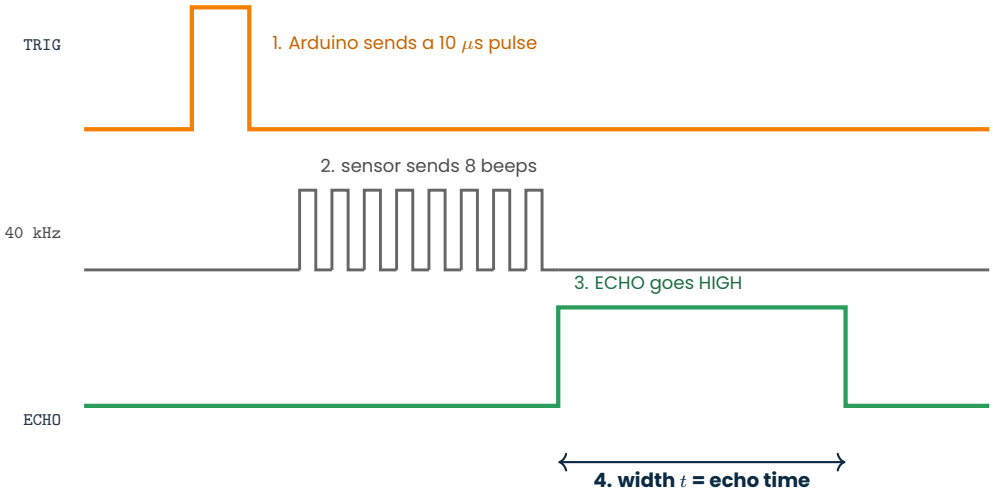
FRONT VIEW



BACK VIEW

Each sensor pin gets its own column, so each pin gets its own wire. Trig is on D4, Echo on D5.

1.2 How the sensor works, in one picture



The sound travels there and back: $d = \frac{v\,t}{2}, \quad v = 0.0343$ cm/μs $\Rightarrow$ $\boxed{d_{\text{cm}} = 0.01715 \cdot t_{\mu s}}$

1.3 The code, one line at a time

1 Name the pins

Distance.ino (top of the file)

```
const int TRIG = 4; // Arduino talks to the sensor here
const int ECHO = 5; // the sensor answers here
```

Naming a pin once means you change the number in one place only. `const` says the value never changes.

2 Set the pin modes and open the Serial link

Distance.ino (setup)

```
void setup() {
  pinMode(TRIG, OUTPUT); // Arduino sends on TRIG
  pinMode(ECHO, INPUT); // Arduino listens on ECHO
  Serial.begin(9600); // open USB link at 9600 baud
}
```

`setup()` runs once at power-up. Both sides of the Serial link must use the same speed: 9600 here and in the Serial Monitor.

3 Send the trigger pulse

Distance.ino (inside loop)

```
digitalWrite(TRIG, LOW);
delayMicroseconds(2); // clean start
digitalWrite(TRIG, HIGH);
delayMicroseconds(10); // the 10 us pulse starts a measurement
digitalWrite(TRIG, LOW);
```

The sensor starts a measurement when TRIG is HIGH for at least 10 μs. `delay()` counts milliseconds, which is 1000 times too slow, so we use `delayMicroseconds()`.

4 Measure how long ECHO stays HIGH

Distance.ino (inside loop)

```
long t = pulseIn(ECHO, HIGH, 30000); // width in microseconds
```

`pulseIn` waits for ECHO to go HIGH, starts a stopwatch, stops it when ECHO falls, and returns the time in μs. After 30 000 μs (30 ms) it gives up and returns 0, which means no echo.

5 Convert time to centimetres

Distance.ino (inside loop)

```
float cm = t * 0.01715; // d = v * t / 2
```

This is the formula from the picture above. `float` keeps the decimals, so 1750 μs gives 30.01 cm.

6 Print and wait

Distance.ino (inside loop)

```
Serial.print("Distance: ");
Serial.print(cm);
Serial.println(" cm");
delay(100); // 10 readings per second
```

`print` stays on the same line, `println` ends the line. The delay lets old echoes die out before the next ping.

SimpleDistance.ino (complete)

```ino
const int TRIG = 4;
const int ECHO = 5;

void setup() {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  long t = pulseIn(ECHO, HIGH, 30000);
  float cm = t * 0.01715;

  Serial.print("Distance: ");
  Serial.print(cm);
  Serial.println(" cm");
  delay(100);
}
```

Serial Monitor 9600 baud

```
Distance: 30.01 cm
Distance: 29.98 cm
Distance: 30.04 cm
Distance: 18.72 cm
Distance: 11.35 cm
Distance: 11.40 cm
Distance: 0.00 cm
Distance: 24.61 cm
```

Typical output. A reading of 0.00 means no echo came back.

1.4 The functions, explained

Every function used in this assignment, with what goes in and what comes out.

`pinMode(pin, mode)`

Does: sets a pin as input or output.
mode: `OUTPUT`, `INPUT` or `INPUT_PULLUP`.
Returns: nothing.
Example: `pinMode(4, OUTPUT);`

`digitalWrite(pin, value)`

Does: puts 5 V (`HIGH`) or 0 V (`LOW`) on an output pin.
Returns: nothing.
Example: `digitalWrite(4, HIGH);`

`delay(ms) / delayMicroseconds(us)`

Does: pauses the program. `delay` counts thousandths of a second, `delayMicroseconds` counts millionths.
Example: `delay(100);` waits 0.1 s.

`pulseIn(pin, state, timeout)`

Does: measures how long a pin stays at `state`.
Returns: time in μs, or 0 on timeout.
Example: `pulseIn(5, HIGH, 30000);`

`Serial.begin(speed)`

Does: opens the USB serial link. Call it once in `setup()`.
speed: 9600 is standard. The Serial Monitor must match.

`Serial.print() / Serial.println()`

Does: sends text or a number to the PC. `println` adds a new line.
Tip: `Serial.print(cm, 1);` shows 1 decimal.
Example: `Serial.println(cm);`

`tone(pin, frequency, duration)`

Does: plays a square wave on a pin so a buzzer beeps.
frequency: in Hz (440 is the note A).
duration: in ms, optional. Without it the tone runs until `noTone`.
Example: `tone(8, 440, 500);`
Watch out: uses Timer2, so PWM on pins 3 and 11 stops working.

`noTone(pin)`

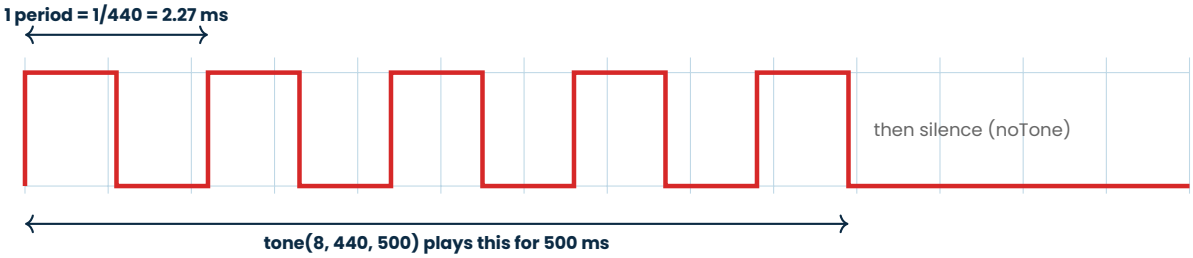
Does: stops the tone on that pin.
Example: `noTone(8);`

`constrain(x, low, high)`

Does: clamps `x` between `low` and `high`.
Example: `constrain(63, 0, 50)` gives 50.

`map(x, inMin, inMax, outMin, outMax)`

Does: rescales a number from one range to another, with integers.
Example: `map(25, 0, 50, 0, 127)` gives 63.

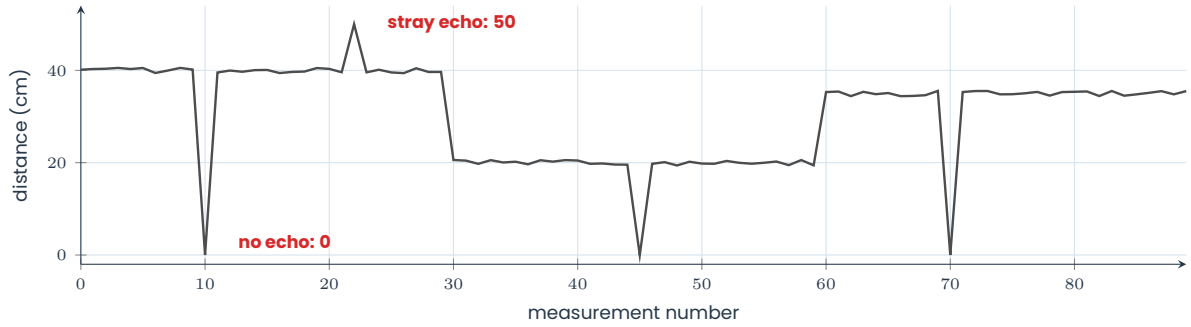


What `tone(8, 440, 500)` puts on pin 8. Not to scale: 500 ms holds about 220 periods.

2 Advanced ultrasonic

Filtering the noise, and rescaling with map()

2.1 Why the numbers jump



A hand held at 40, then 20, then 35 cm. Small wobble is normal. The spikes are bad readings.

Resolution	about 3 mm: small random wobble in every reading
No echo	soft cloth, a sloping surface, or too far: `pulseIn` returns 0
Stray echo	a wall or table edge answers instead of your target

2.2 Three filtering methods

1 Method 1: the mean (average)

$$\bar{x} = \frac{x_1 + x_2 + x_3}{3}$$

Take 3 readings, add them, divide by 3. Smooth, but one bad reading drags the result: $(40 + 40 + 0)/3 = 26.7$.

mean of 3

```
float readMean3() {
  float sum = 0;
  for (int i = 0; i < 3; i++) {
    sum += readCm(); // your function
    delay(30);
  }
  return sum / 3;
}
```

2 Method 2: the median

sort x_1, x_2, x_3, keep the middle one

Take 3 readings and keep the middle value. A single spike is always the largest or smallest, so it is thrown away: median of $(40, 40, 0)$ is 40.

median of 3

```
float median3(float a, float b, float c) {
  float t;
  if (a > b) { t = a; a = b; b = t; }
  if (b > c) { t = b; b = c; c = t; }
  if (a > b) { t = a; a = b; b = t; }
  return b; // the middle one
}
```

3 Method 3: exponential smoothing

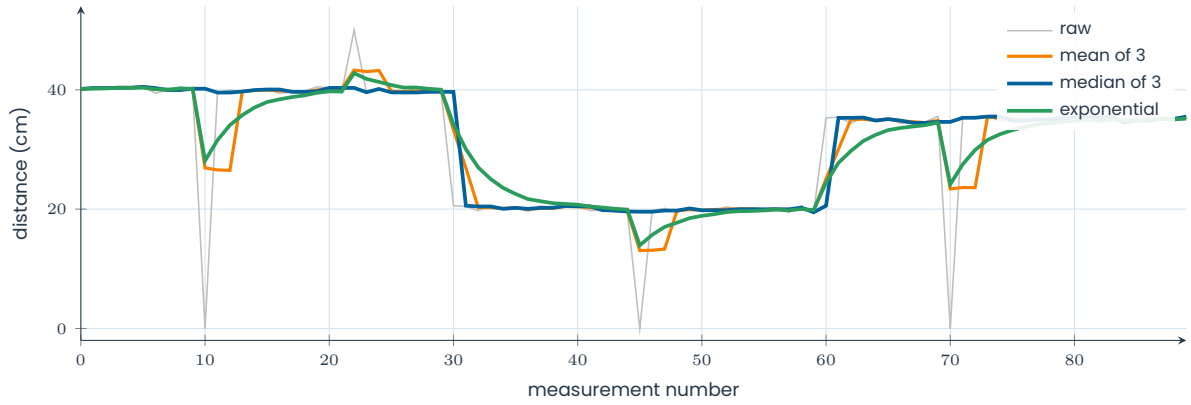
$$s_{\text{new}} = 0.7\, s_{\text{old}} + 0.3\, x$$

Keep one running value. Each new reading moves it 30% of the way. Needs no history, but it reacts slowly: a smaller 0.3 means smoother and slower.

exponential smoothing

```
float smooth = 0;
bool first = true;

float ema(float x) {
  if (first) { smooth = x; first = false; }
  smooth = 0.7 * smooth + 0.3 * x;
  return smooth;
}
```



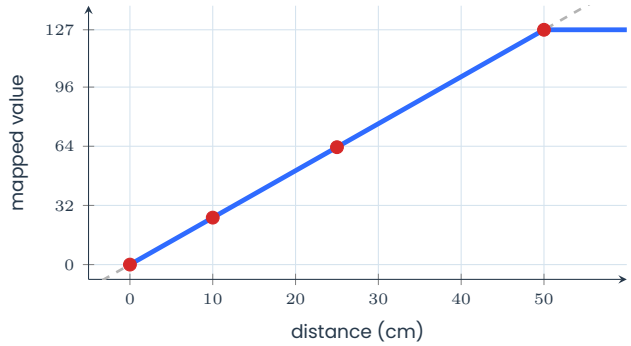
Same data, three filters. The median removes every spike. The mean smears each spike over 3 readings. The exponential curve is smooth but lags behind the hand.

Method	Good at	Weak at	Memory
Mean	smoothing small wobble	spikes leak in	3 readings
Median	deleting spikes	small wobble remains	3 readings
Exponential	small wobble, very cheap	reacts slowly	1 number

Best practice

Combine them: median first to delete spikes, then exponential smoothing to calm the wobble. That is the filter chain used in the Air Theremin.

2.3 Rescaling with map()



Distance	map() result
0 cm	0
10 cm	25
25 cm	63
50 cm	127

Beyond 50 cm the dashed line would keep climbing. We clamp it at 127 with `constrain()`.

1 The idea: one range onto another

$$\text{out} = \frac{(x - \text{inMin})\,(\text{outMax} - \text{outMin})}{\text{inMax} - \text{inMin}} + \text{outMin}$$

2 Fill in our numbers

$$\text{out} = \frac{(x - 0)\,(127 - 0)}{50 - 0} + 0 = \frac{127}{50}\,x = 2.54\,x$$

Every centimetre adds 2.54 steps. Why 127? It is the largest value of 7 bits (MIDI range 0 to 127).

3 Two traps

Integers only: `map` drops the decimals, so `map(10, 0, 50, 0, 127)` is 25, not 25.4.

No clamping: `map(70, 0, 50, 0, 127)` gives 177. Always `constrain` first.

map + constrain

```
float d = constrain(filtered, 0, 50); // keep inside 0..50
int value = map((int)(d * 10), 0, 500, 0, 127); // x10 keeps one decimal of precision
```

2.4 The full advanced sketch

FilteredDistance.ino

```
const int TRIG = 4;
const int ECHO = 5;

float smooth = 0;
bool first = true;
float h1 = 0, h2 = 0; // the last two readings

float readCm() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long t = pulseIn(ECHO, HIGH, 30000);
  if (t == 0) return -1; // no echo
  return t * 0.01715;
}

float median3(float a, float b, float c) {
  float t;
  if (a > b) { t = a; a = b; b = t; }
  if (b > c) { t = b; b = c; c = t; }
  if (a > b) { t = a; a = b; b = t; }
  return b;
}

void setup() {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  Serial.begin(9600);
}

void loop() {
  float raw = readCm();
  if (raw < 0) { delay(60); return; } // skip bad readings

  float med = median3(h1, h2, raw); // method 2
  h1 = h2; h2 = raw;
  if (first) { smooth = med; first = false; }
  smooth = 0.7 * smooth + 0.3 * med; // method 3

  float d = constrain(smooth, 0, 50);
  int value = map((int)(d * 10), 0, 500, 0, 127);

  Serial.print(raw); Serial.print(",");
  Serial.print(smooth); Serial.print(",");
  Serial.println(value);
  delay(60);
}
```

Prints three comma-separated numbers per line: raw, filtered, mapped. The Serial Plotter can draw all three.

3 Your assignments

One required, one extra

Assignment: a simple ultrasonic distance sensor

1. Wire the HC-SR04 as in section 1.1 (Trig on D4, Echo on D5).
2. Write the sketch yourself. Do not copy the complete listing.
3. Print the distance in centimetres to the Serial Monitor, one line per reading.
4. Check it with a ruler at 10, 20 and 30 cm and write down your error.

Hand in

- the `.ino` file
- a photo of your wiring
- a screenshot of the Serial Monitor
- a 3-row table: ruler, measured, error

Marking (10 points)

Correct wiring	2
Trigger pulse and `pulseIn`	3
Correct conversion to cm	2
Clean Serial output	1
Ruler check table	2

Extra: filtered distance, mapped to 0 to 127

1. Start from your required sketch.
2. Add a filter. Use at least one of the three methods, and explain your choice in 2 lines.
3. Clamp the distance to 0 to 50 cm with `constrain()`.
4. Convert to 0 to 127 with `map()`.
5. Print **raw, filtered, mapped** on one line, separated by commas.

Hand in

- the `.ino` file
- a Serial Plotter screenshot with the 3 curves
- the values you get at 0, 25 and 50 cm

Marking (10 extra points)

Working filter	4
`constrain` and `map`	3
Three-value output	1
Plotter screenshot and explanation	2

If it does not work

Symptom	Likely cause	Check
Always 0.00	no echo, or Trig and Echo swapped	Trig on D4, Echo on D5, 5V and GND connected
Garbage characters	speed mismatch	Serial Monitor set to 9600 baud
Wrong by a factor 2	forgot the “there and back”	use 0.01715, not 0.0343
Mapped value above 127	no clamping	call `constrain` before `map`
Mapped value stuck at 0	integer division	multiply the distance by 10 before casting to int